



A PROJECT REPORT ON

TEACHER'S PROFILE MANAGEMENT SYSTEM

*Submitted in partial fulfilment of the requirements for the award of the degree
of Master of Computer Applications (MCA)*

Department of Computer Science / Applications (MCA), J.D. Women's College

Guide:

Mr. Anant Kumar

Submitted By:

Name: Arpi kumari

College Roll No.: 18

Univ. Exam Roll No.: 2520840170004

Univ. Regd. No.: 192082114008003

Session: 2024-2026



PATLIPUTRA UNIVERSITY, PATNA

ACKNOWLEDGEMENT

The success and outcome of my project, **Teacher's Profile Management system**, required a lot of guidance and assistance from many people and I am extremely privileged to have completed my project due to their kind support. All that I have done is only due to such supervision and assistance and I am very grateful to them.

I offer sincere gratitude to the faculty of **J. D. Women's College, Patna**, for their sincere guidance and motivation, to uphold my project in all its theoretical and at the practical side too.

I specially owe my thanks to our project guide for his constant support and guidance throughout the course of my work. His sincerity, thoroughness and positive mindset have been a constant source of inspiration for me. I am very grateful to my project coordinator **Mr. Anant Kumar**; he has always encouraged me to work independently and allowed my idea to flourish unrestricted. It is only his cognizant efforts that my endeavors have seen light of the day.

DECLARATION

This is to certify that the dissertation / project report entitled **“Teacher’ Profile Management System”** is done by me work carried out for the partial fulfilment of the requirements for the award of the degree of Master of Computer Applications (MCA) under the guidance of **Mr. Anant Kumar**. The matter embodied in this project work has not been submitted earlier for award of any degree or diploma to the best of my knowledge and belief.

*Signature of the Student: Arpi
kumari*

Univ. Regd. No.: 192082114008003

Univ. Roll No.: 2520840170004

Course: MCA

Session: 2024-2026

CERTIFICATE OF ORIGINALITY

This is to certify that the project report entitled "**Teacher's Profile management System**" submitted to Patliputra University, Patna in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA) through the Department of Computer Applications, J. D. Women's College, is original work carried out by **Arpi Kumari**.

Univ. Regd. No.: 192082114008003

Univ. Roll No.: 2520840170004

Course: MCA

Session: 2024-2026

The matter embodied in this project is genuine work done by the student under the guidance of Mr. Anant Kumar and has not been submitted earlier to this University or to any other University / Institute for the fulfillment of any course of study.

Signature of the Student:

Signature of the Guide:

Date:

Date:

Name:

Name:

Course:

Designation:

Session:

Table of Contents

S.NO.	Topics	Page No.
01.	Synopsis	7
02.	Introduction	8-9
03.	Objective of the project	10
04.	System Analysis	11
4.1	Identification of need	11
4.2	Problem Definition	11
4.3	Preliminary Investigation	12
4.4	Feasibility Study	12
4.4.1	Technical Feasibility	12
4.4.2	Economical Feasibility	12
4.4.3	Operational Feasibility	12
4.5	System Planning Scheduling chart:- PERT chart, Gantt chart	13-14
4.6	Software and Hardware Requirement Specification	14-15
4.7	DFD	15
4.8	E-R Diagram	16
4.9	Use Case Diagram	17
4.10	Class Diagram	18
05.	System Design	19
5.1	Number of Modules and their Description	20
5.2	Data Structure/ Data Base Design/ Data Dictionary	21-22
5.3	Process Logic of each module	23-24
06.	GUI Interface & Source Code, Report Generation	25-130
07.	System Testing	131
7.1	Code Efficiency	132
7.2	Optimization of Code	132-133
7.3	Validation Check	134-135
08.	Implementation And Maintenance	136-137
09.	System Security measures	138
10.	Cost Estimation of the project	139-140
11.	Future Scope and further enhancement of the project	141-142
12.	Conclusion	143
13.	Bibliography	144
14.	Glossary	145-146

Introduction

Educational institutions manage a large amount of teacher-related information such as name, qualifications, experience, email, and photographs. Maintaining these records manually often leads to data inconsistency, errors, and difficulty in retrieving information when needed.

To address these challenges, the **Teacher Management System** has been developed as a web-based application using the *Django Framework*, *Bootstrap*, and *SQLite* database.

This system provides an intuitive and responsive interface for administrators to manage teacher records efficiently. Users can log in securely, add new teachers, upload photos, download or upload records through Excel files, and view necessary statistics on a clean dashboard layout. The project follows the Django **Model–View–Template (MVT)** architecture, ensuring modularity, easy maintenance, and scalability.

The Teacher Management System brings digital transformation to educational record management, making processes faster, more accurate, and automated.

PROBLEM STATEMENT

In many educational institutions, teacher information such as personal details, qualifications, experience, and contact data is still maintained manually using registers, handwritten files, or basic spreadsheets.

This traditional method of data management is time-consuming, error-prone, and inefficient when the number of staff increases.

Additionally, searching for a specific teacher's profile, updating information, generating reports, or maintaining photo records manually becomes very challenging.

There is also no automated way to track the number of teachers added, deleted, or currently available in the institution.

Lack of centralized digital data storage leads to:

- 1. Difficulty in quick data retrieval**
- 2. Risk of data loss or duplication**
- 3. Inability to maintain photos and documents properly**
- 4. Inconsistent record formats**
- 5. No role-based secure access**
- 6. No import/export support for bulk operations**

Hence, a modern, reliable, and secure system is required to simplify teacher data management using a web-based interface with database support and automated functionalities.

Objective

The main objectives of the Teacher Management System are:

Functional Objectives

- To create a secure login system for administrators
- To allow adding, updating, deleting, and viewing teacher profiles
- To support photo upload for each teacher
- To enable Excel data export (download)
- To enable Excel data import (upload)
- To display teacher statistics on the dashboard

Technical Objectives

- To implement Django MVT architecture
- To use SQLite for secure and structured storage of teacher records
- To develop a responsive UI using Bootstrap
- To ensure modularity and easy scalability

Operational Objectives

- To reduce manual paperwork
- To increase accuracy and consistency in teacher records
- To simplify data search and retrieval

EXISTING SYSTEM VS PROPOSED SYSTEM

Existing System

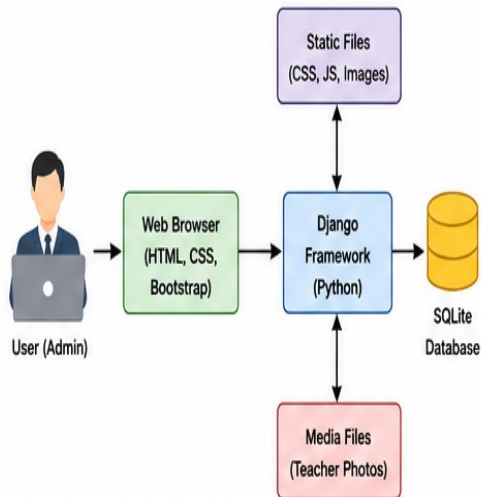
- Manual entry of teacher data
- High risk of data duplication
- Inefficient photo/document management
- No centralized database
- Time-consuming search
- No real-time statistics
- No Excel bulk upload/download

Proposed System

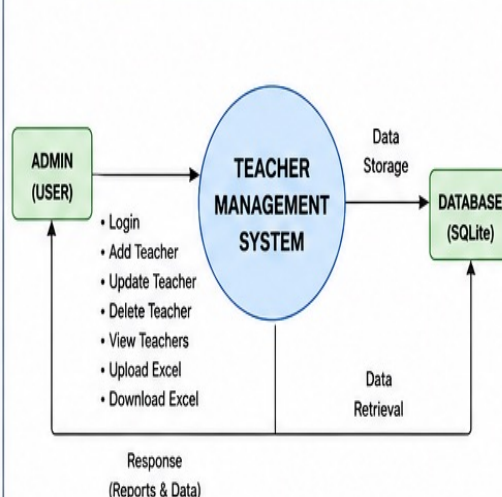
- Fully automated web-based system
- Centralized and secure data storage
- Quick retrieval and modification
- Photo upload support
- Dashboards and analytics
- Bulk Excel operations
- Role-based access via login system
- User-friendly responsive UI

SYSTEM PLANNING DIAGRAMS – TEACHER MANAGEMENT SYSTEM

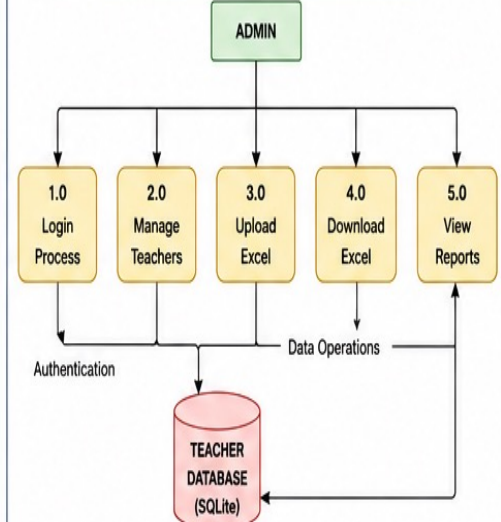
1. SYSTEM ARCHITECTURE DIAGRAM



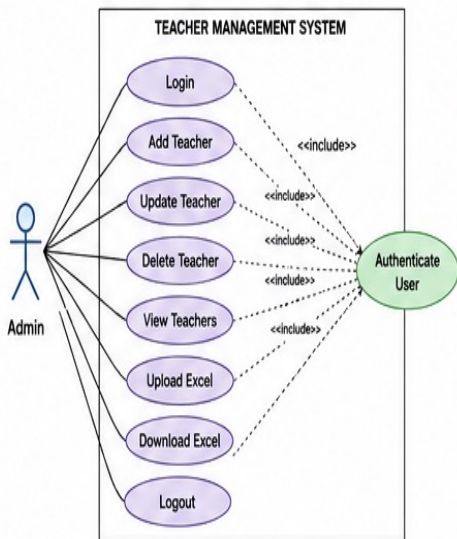
2. CONTEXT DIAGRAM (DFD LEVEL 0)



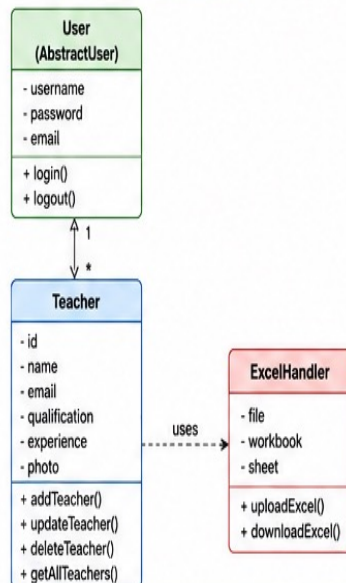
3. DATA FLOW DIAGRAM (DFD LEVEL 1)



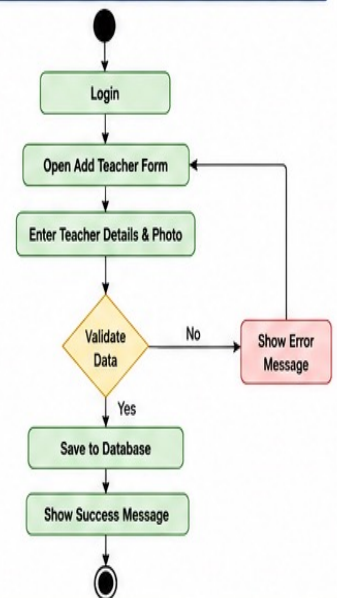
4. USE CASE DIAGRAM



5. CLASS DIAGRAM



6. ACTIVITY DIAGRAM (ADD TEACHER PROCESS)



SYSTEM SUMMARY

- Admin logs in to the system.
- Admin can Add, Update, Delete, View Teachers.
- Admin can Upload Excel file to import teachers in bulk.
- Admin can Download Excel file of teachers.
- All data is stored in SQLite Database.
- Teacher photos are stored in Media Folder.



Explanation Of Flow Diagram

1. Login

- Admin enters username and password.
- If correct → goes to **Dashboard**
- If wrong → error message and stays on login page.

2. Dashboard

Shows:

- Total active teachers
- Total deleted teachers
- Quick buttons:
 1. Add Teacher
 2. View Teachers
 3. Logout

3. Add Teacher

Admin fills the form:

- Name
- Email
- Phone
- Subject
- Upload Photo

Data is saved into:

- **Teacher model**
- Photo goes into **media/teachers/**

4. View Teachers

Displays:

- List of teachers
- Their photos
- Edit button
- Delete button

5. Edit Teacher

Admin can update:

- Details
- Photo

Updated data saves back to database.

6. Delete Teacher

When deleting:

- Teacher record removed from Teacher table
- Name stored in **DeletedTeacher model** for count tracking

7. Logout

Session ends and user returns to login page.

SOFTWARE & HARDWARE REQUIREMENTS

Hardware Requirements:--

- Minimum 2 GB RAM (4 GB recommended)
- 1.5 GHz Processor or above
- 200 MB system storage for project
- Laptop/Desktop

Software Requirements

- Python 3.10+
- Django 5+
- SQLite Database
- Visual Studio Code (recommended IDE)
- Browser (Chrome/Edge/Firefox)
- Bootstrap CDN
- OpenPyXL for Excel processing
- Pillow library for image upload

FEASIBILITY STUDY

1. Technical Feasibility

All the required technologies such as Python, Django, SQLite, and Bootstrap are open-source and easy to install. No advanced hardware is required.

2. Operational Feasibility

The system is easy to operate with simple menu options and a clean dashboard. Minimal training is required.

3. Economic Feasibility

The entire project is based on free tools and libraries, making it cost-effective for institutions.

SYSTEM DESIGN

A. MVT Architecture

- ❖ **Model:** Defines Teacher model
- ❖ **View:** Handles business logic
- ❖ **Template:** Renders HTML pages

B. Data Flow Diagram (Description)

Level 0 DFD

User → Login System → Teacher Management → Database

Level 1 DFD

1. Login Operation
2. Add Teacher
3. Delete Teacher
4. Update Teacher
5. View Teacher
6. Import/Export Excel

C. UML DIAGRAMS (Description)

Use Case Diagram

Actors:

- ❖ Admin

Use Cases:

- ❖ Login
- ❖ Manage Teachers
- ❖ Upload Excel
- ❖ Download Excel
- ❖ View Dashboard

Class Diagram

Classes:

- ❖ Teacher
- ❖ User
- ❖ Excel Handler

Relationships:

- ❖ Teacher → SQLite (One-to-Many Records)
- ❖ User → System (Authentication)

MODULE DESCRIPTION

1. Login Module

- Allows admin login
- Secured through Django authentication
- Redirects to dashboard

2. Dashboard Module

- Displays total teachers
- Displays added/deleted count
- Shows quick navigation cards

3. Teacher Management Module

Features:

- Add Teacher
- Upload Photo
- Update Teacher
- Delete Teacher
- View Profile
- List All Teachers

4. Excel Import Module

- Upload Excel (.xlsx) file
- Validates data
- Inserts records in bulk

5. Excel Export Module

- Downloads all teacher data in Excel format
- Used for backups

6. Photo Upload Handling

- Stores photos in /media/teachers/
- Uses Django's Pillow library

TESTING

Unit Testing

- Test each form input
- Test database insert/update/delete

Integration Testing

- Verify flow from login → dashboard → data management

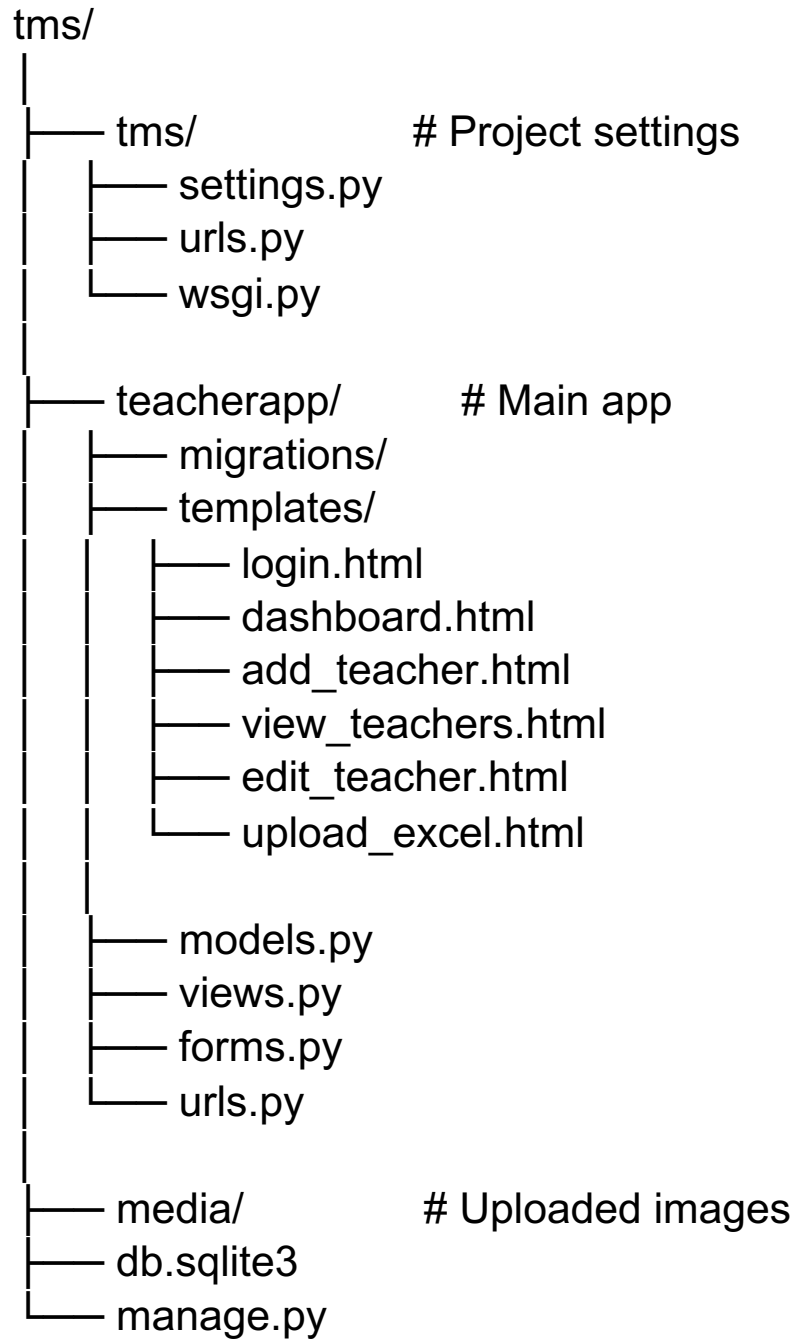
User Acceptance Testing

- Verify UI usability
- Check accuracy of Excel file import/export

Test Cases Example:-

Test Case	Input	Expected Output
Login	valid username/pass word	Redirect to dashboard
Add Teacher	Name, Email, Photo	Record saved
Excel Upload	Valid file	Bulk records added
Photo Upload	.jpg file	Photo visible in view page

PROJECT STRUCTURE



INSTALLATION COMMANDS

```
pip install django  
pip install pillow  
pip install openpyxl
```

SETTINGS CONFIGURATION

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'teacherapp',  
]
```

MEDIA CONFIG:

```
import os

MEDIA_URL = '/media/'
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

MODEL (DATABASE STRUCTURE):

```
from django.db import models

class Teacher(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    qualification = models.CharField(max_length=100)
    experience = models.CharField(max_length=50)
    photo = models.ImageField(upload_to='teachers/', blank=True,
                              null=True)

    def __str__(self):
        return self.name
```

FORMS:

```
from django import forms
from .models import Teacher

class TeacherForm(forms.ModelForm):
    class Meta:
        model = Teacher
        fields = '__all__'
```

VIEWS (CORE LOGIC)

```
from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login, logout
from django.contrib import messages
from .models import Teacher
from .forms import TeacherForm
import openpyxl
from django.http import HttpResponse
```

LOGIN

```
def login_page(request):
    if request.method == "POST":
        uname = request.POST.get('username')
        pwd = request.POST.get('password')

        user = authenticate(username=uname, password=pwd)

        if user:
            login(request, user)
            return redirect('dashboard')
        else:
            messages.error(request, "Invalid Credentials")

    return render(request, "login.html")
```

DASHBOARD

```
def dashboard(request):
    total_teachers = Teacher.objects.count()
    total_inserted = Teacher.objects.count()
    total_deleted = request.session.get("deleted", 0)

    return render(request, "dashboard.html", {
        "total_teachers": total_teachers,
        "total_inserted": total_inserted,
        "total_deleted": total_deleted
    })
```

ADD TEACHER

```
def add_teacher(request):
    form = TeacherForm()
    if request.method == "POST":
        form = TeacherForm(request.POST, request.FILES)
    if form.is_valid():
        form.save()
    return redirect('view_teachers')
    return render(request, "add_teacher.html", {"form": form})
```

VIEW TEACHERS

```
def view_teachers(request):
    teachers = Teacher.objects.all()
    return render(request, "view_teachers.html", {"teachers": teachers})
```

EDIT

```
def edit_teacher(request, id):
    teacher = Teacher.objects.get(id=id)
    form = TeacherForm(instance=teacher)

    if request.method == "POST":
        form = TeacherForm(request.POST, request.FILES, instance=teacher)
    if form.is_valid():
        form.save()
    return redirect('view_teachers')

    return render(request, "edit_teacher.html", {"form": form})
```



```

# DELETE
def delete_teacher(request, id):
    teacher = Teacher.objects.get(id=id)
    teacher.delete()

    request.session["deleted"] = request.session.get("deleted", 0) + 1
    return redirect('view_teachers')


# LOGOUT
def logout_user(request):
    logout(request)
    return redirect('login')


# EXPORT EXCEL
def download_excel(request):
    wb = openpyxl.Workbook()
    sheet = wb.active
    sheet.title = "Teachers"

    sheet.append(["Name", "Email", "Qualification", "Experience"])

    for t in Teacher.objects.all():
        sheet.append([t.name, t.email, t.qualification, t.experience])

    response = HttpResponse(
        content_type='application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'
    )
    response['Content-Disposition'] = 'attachment; filename=teachers.xlsx'
    wb.save(response)
    return response

```

URLS:

```
from django.contrib import admin
from django.urls import path
from teacherapp import views
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),

    path("", views.login_page, name='login'),
    path('dashboard/', views.dashboard, name='dashboard'),

    path('add/', views.add_teacher),
    path('view/', views.view_teachers, name='view_teachers'),

    path('edit/<int:id>/', views.edit_teacher),
    path('delete/<int:id>/', views.delete_teacher),

    path('logout/', views.logout_user),

    path('download/', views.download_excel),
    path('upload/', views.upload_excel),
]

urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)
```

TEMPLATES :

- ❖ login.html
- ❖ dashboard.html
- ❖ add_teacher.html
- ❖ view_teachers.html
- ❖ edit_teacher.html

WORKING FLOW (TECHNICAL EXPLANATION)

1. User opens login page
2. Authentication using Django auth system
3. Redirect to dashboard
4. Dashboard fetches data using ORM
5. CRUD operations manage teacher table
6. Images stored in MEDIA folder
7. Excel import/export handled using openpyxl
8. Templates render data using Django MVT

FEATURES SUMMARY

- ✓ Secure Login System
- ✓ Dashboard with counters
- ✓ CRUD operations
- ✓ Image upload support
- ✓ Excel import/export
- ✓ Bootstrap UI
- ✓ Session tracking for deletes

Conclusion

The documentation of the Face Recognition Attendance System serves as a comprehensive reference that outlines every critical aspect of the project - from initial planning and system design to development, testing, and deployment. It ensures that all stakeholders, including developers, testers, teachers, students, and project evaluators, have a clear understanding of the system's functionality, architecture, and workflow.

Through detailed modules such as system overview, data flow diagrams, ER diagrams, use case analysis, and process logic, the documentation demonstrates the technical and logical structure of the application. It highlights how the system addresses real-world problems in manual attendance management by offering an automated, contactless, secure, and time-efficient platform using OpenCV and face_recognition libraries.

Additionally, the inclusion of requirement specifications, validation checks, security measures like liveness detection and AES-256 encryption, and future enhancement plans adds value by promoting scalability and maintainability. The clear articulation of processes, supported with flowcharts and database design, contributes to better project communication and implementation.

In conclusion, this documentation not only validates the project's design and execution but also acts as a vital tool for maintenance, troubleshooting, future upgrades, and training. The system successfully eliminates proxy attendance, reduces faculty workload, and provides accurate attendance records with >95% recognition accuracy. It reflects a well-organized and professionally managed software development lifecycle, aligning with industry best practices and academic requirements.

Bibliography

Books and Academic References

1. E. Balagurusamy, Programming in Python, McGraw Hill Education, 2019.
2. Ramez Elmasri & Shamkant B. Navathe, Fundamentals of Database Systems, Pearson Education, 7th Edition.
3. Roger S. Pressman, Software Engineering: A Practitioner's Approach, McGraw Hill, 8th Edition.
4. Ian Sommerville, Software Engineering, Pearson Education, 10th Edition.
5. Richard Szeliski, Computer Vision: Algorithms and Applications, Springer, 2nd Edition.

Websites and Online Resources

1. <https://www.python.org> - Official Python Documentation
2. <https://docs.opencv.org> - Official OpenCV Library Documentation
3. https://github.com/ageitgey/face_recognition - Face Recognition Library by Adam Geitgey
4. <https://dev.mysql.com/doc> - Official MySQL 8.0 Documentation
5. <https://flask.palletsprojects.com> - Flask Web Framework Documentation
6. <https://www.w3schools.com> - HTML, CSS, and JavaScript Reference
7. <https://developer.mozilla.org> - MDN Web Docs for HTML5 and JavaScript
8. <https://draw.io> - Used to create system DFD, ER diagrams and flowcharts
9. IEEE Xplore Digital Library - Research papers on Face Recognition Algorithms

Glossary

Term	Definition
Attendance System	A software application that uses facial recognition to automatically mark and manage student attendance.
Frontend	The user-facing part of the application built using HTML5, CSS3, JavaScript, and Bootstrap for teacher/admin dashboard.
Backend	The server-side logic written in Python using Flask framework that processes face detection, encoding, and database operations.
API	Application Programming Interface - Flask endpoints used to allow communication between frontend and MySQL database.
Face Encoding	A 128-dimension numerical vector generated from facial landmarks that uniquely represents a person's face. Used for matching.
HOG	Histogram of Oriented Gradients - A feature descriptor used by dlib for fast and accurate face detection on CPU.
OpenCV	Open Source Computer Vision Library - Used for webcam access, image capture, and pre-processing like grayscale conversion.

MySQL	A relational database management system used to store student details, face encodings, and attendance records in tables.
Liveness Detection	A security technique using blink detection and texture analysis to verify a real person is present, preventing photo/video spoofing.
Euclidean Distance	A mathematical method to calculate similarity between two face encodings. Distance < 0.6 means it's a match.
User Authentication	The process of verifying identity of Teacher/Admin using username and bcrypt hashed password. Students are authenticated using facial recognition.
Session	Temporary data stored on the server to track Teacher/Admin login activity. Auto-expires after 30 minutes of inactivity.
Client-Side	Operations executed in the Teacher's browser using HTML/JavaScript. Example: Webcam access, displaying live video feed.
Server-Side	Operations processed on the server using Python-Flask. Example: Face detection, generating face encoding, matching with MySQL.
Flask	A Python micro web framework used to create the backend API endpoints for handling attendance requests and database operations.
AES-256	Advanced Encryption Standard used to encrypt 128-dimension face encodings before storing in MySQL database for biometric data security.

THANK YOU